2023

Key-Value-Datenbanken

MIM9 DATENBANKEN / FORTGESCHRITTENE ASPEKTE

JEROME HELZEL

Inhaltsverzeichnis

Was ist NoSQL? 1

SQL vs. NoSQL 1

SQL 2

NoSQL 2

Anwendungsbereiche für SQL & NoSQL 2

SQL-Datenbanken 2

NoSQL-Datenbanken 3

Einführung in Key-Value-Datenbanken & Redis 3

Datentypen in Redis 3

Strings 4

Lists 4

Sets 4

Hashes 4

Schlüssel mit Lebensdauer 4

Praxis 5

Installation 5

Node.js 5

Docker 5

Einführung in Docker und Erstellung einer Redis-Datenbank 6

Grundlegende Redis Befehle 7

Strings + Allgemeine Befehle 8

Ablaufende Schlüssel 11

Lists 13

Sets 14

Hashes 15

Weitere Befehle 16

Literatur 17

Was ist NoSQL?

NoSQL = „Not only SQL“

NoSQL bezeichnet eine Gruppe von Datenbanken, die im Gegensatz zu relationalen Datenbanken nicht auf einem tabellarischen Datenmodell basieren. Die verschiedenen Arten von NoSQL-Datenbanken haben unterschiedliche Datenmodelle und -strukturen.

1. **Document Store**: Speichert Daten in flexiblen, dokumentenorientierten Formaten wie JSON oder XML. Dokumente können unterschiedliche Strukturen haben, was sie für komplexe Datenmodelle und Änderungen am Schema gut geeignet macht.
2. **Key-Value Store**: Speichert Daten als Schlüssel-Wert-Paare, bei denen jeder Schlüssel einen eindeutigen Wert repräsentiert. Es gibt keine festgelegte Struktur für den Wert, was Flexibilität bietet, aber komplexe Abfragen erschwert.
3. **Wide-Column Store**: Organisiert Daten in Spalten anstelle von Zeilen, wodurch eine flexible und skalierbare Speicherung ermöglicht wird. Es eignet sich besonders für Anwendungen mit großen Datensätzen und variablen Attributen.
4. **Graph Store**: Speichert Daten in Form von Knoten und Kanten, wodurch komplexe Beziehungen zwischen den Datenmodellen dargestellt werden können. Dies macht Graphdatenbanken ideal für Anwendungen wie soziale Netzwerke oder Wissensgraphen.

SQL vs. NoSQL

SQL-Datenbanken arbeiten nach einem strukturierten Ansatz mit einem tabellenbasierten Datenmodell. Sie haben klare Regeln für die Datenmodellierung und die Datenkonsistenz. Im Gegensatz dazu arbeiten NoSQL-Datenbanken unstrukturiert und flexibel. Die Datenmodellierung und -konsistenz sind nicht so streng definiert.

Ein weiterer Unterschied ist die Skalierung. SQL-Datenbanken bieten vertikale Skalierung, d.h. die Möglichkeit, die Hardware zu verbessern. NoSQL-Datenbanken hingegen bieten die Möglichkeit der horizontalen Skalierung. Das bedeutet, dass zusätzliche Server hinzugefügt werden können, um die Last auf mehrere Server zu verteilen.

Der Vorteil von SQL-Datenbanken liegt in der festen Sprache für die Manipulation von Daten, was die Fehlerbehandlung erleichtert. Der Nachteil von NoSQL-Datenbanken ist, dass es keine feste Sprache für die Manipulation von Daten gibt und in der Regel eine hauseigene Query Language verwendet wird. Allerdings sind NoSQL-Datenbanken besser geeignet, wenn Daten nicht gut in einer Tabelle darzustellen sind, häufige oder variable Schema-Änderungen vorliegen oder Daten in vielen unterschiedlichen Formen gespeichert werden müssen. Außerdem können große Datenmengen ohne ACID-Zwang verwaltet werden, da das Datenformat von NoSQL-Datenbanken sowieso in der Applikation verwendet wird. ACID würde bedeuten, dass die Daten atomar, konsistent, isoliert und dauerhaft sind.

Zusammengefasst weisen beide Datenbanken folgende Eigenschaften auf:

SQL

1. Bietet eine strukturierte Herangehensweise
    a. Die Struktur ist von vorneherein fest vorgegeben
2. Speichert Daten in einem tabellenbasiertem Datenmodel
    a. Dieses Modell ist fest vorgegeben und kann nicht geändert werden.
3. Ist vertikal skalierbar
    a. Bedeutet, dass die Datenbankleistung durch Verbesserung der Hardware, wie z.B. Hinzufügen von mehr RAM oder CPU-Kernen, verbessert werden kann.

NoSQL

1. Bietet eine unstrukturierte Herangehensweise an die Speicherung von Daten
    a. Eine genaue Vorgabe zur Struktur, ist nicht vorgegeben
2. Flexibles Datenmodel
    a. Das Datenmodell kann innerhalb einer Datenbank unterschiedlich sein, oder variieren. Man muss sich zu keinem Zeitpunkt festlegen.
3. Horizontal skalierbar
    a. Bedeutet, dass leicht neue Server hinzugefügt werden können, was die Last verteilt.
    b. Da die ID der einzige Diskriminator für die Daten ist, kann Sharding sehr einfach angewendet werden, wodurch die Datenbank sehr leicht horizontal skalierbar ist.

Anwendungsbereiche für SQL & NoSQL

Es gibt keine generelle Aussage darüber, ob ein Datenbankkonzept besser oder schlechter als das andere ist. Es hängt vielmehr davon ab, welches Konzept besser für ein bestimmtes Projekt geeignet ist. Je nach Anforderungen kann entweder eine SQL- oder eine NoSQL-Datenbank die bessere Wahl sein.

Es gibt gute Gründe, NoSQL für ein Projekt zu verwenden. Zum Beispiel wenn:

1. die Daten sich nicht gut in tabellarischer Form darstellen lassen,
2. das Schema häufig ändert oder variiert,
3. die Daten in unterschiedlicher Form gespeichert werden müssen, oder
4. es sich um sehr große Datenmengen handelt, auf die keine ACID-Beschränkungen angewendet werden müssen.

Wenn die eben genannten Gründe nicht oder zum großen Teil nicht zutreffen, spricht das eher für die Verwendung einer SQL-Datenbank.

Einige typische Anwendungsbereiche sind:

SQL-Datenbanken

- **E-Commerce**: Produkte, Bestellungen, ...
- **Finanzwesen**: Transaktionsdaten, Kundeninformationen, ...
- **Gesundheitswesen**: Behandlungsverläufe, Patientendaten, ...

NoSQL-Datenbanken

- **Social Media**: Benutzerprofile und Aktivitätsströme
- **IoT**: Sensor-, Gerätedaten, Ereignisse, ...
- **CMS**: Aufgrund flexibler Datenmodellierung und Skalierbarkeit

Einführung in Key-Value-Datenbanken & Redis

Key-Value-Datenbanken bestehen aus einem einfachen Datenmodell und einer simplen Struktur. Jeder Datensatz besteht lediglich aus einem Schlüssel und einem Wert. Der Wert wird in der Regel als String gespeichert. Aufgrund dieser simplen Struktur sind Key-Value-Datenbanken sehr schnell und effizient. Dieses Datenmodell kann man sich vorstellen, wie ein JSON-Objekt.

```
{
    "fantasy": [
      {
        "title": "Harry Potter and the Sorcerer's Stone",
        "author": "J.K. Rowling",
        "publicationYear": 1997
      },
      {
        "title": "The Hobbit",
        "author": "J.R.R. Tolkien",
        "publicationYear": 1937
      }
    ],
    "scienceFiction": [
      {
        "title": "Dune",
        "author": "Frank Herbert",
        "publicationYear": 1965
      },
      {
        "title": "1984",
        "author": "George Orwell",
        "publicationYear": 1949
      }
    ]
}
```

Redis ist eine der bekanntesten und meistverwendeten Key-Value-Datenbanken. Da Redis auf einem In-Memory-Datenmodell basiert, ist sie besonders schnell und effizient. Das bedeutet, dass Daten im Speicher (RAM) statt auf der Festplatte gespeichert werden. Es ist allerdings auch möglich Redis so zu konfigurieren, dass die Daten persistent sind, also auf einer Festplatte gespeichert werden. Aufgrund dieser Eigenschaften wird Redis oft in Zusammenarbeit mit einer anderen persistenten Datenbank als Cache oder Zwischenspeicher genutzt. Dies führt dazu, dass die Antwortzeit eines Servers um ca. das 10-fache verringert werden kann.

Datentypen in Redis

In Redis lassen sich Daten in verschiedener Form speichern.

Strings

- **Schlüssel**: name
- **Wert**: “Max Mustermann“

Bei Strings kann eine beliebige Zeichenkette unter einem Schlüssel gespeichert.
Wichtig hier ist zu verstehen, dass auch Zahlen als Strings gespeichert werden!

Neben dem Erstellen, Bearbeiten und Löschen eines Schlüssels, stellt Redis weitere Operation zur Verfügung. Die wichtigsten sind:

1. Zu überprüfen, ob ein Schlüssel existiert (gilt für alle Datentypen)
2. Die Ausgabe aller Schlüssel, die einem bestimmten Pattern entsprechen (gilt für alle Datentypen)
3. Für reine Zahlen als Werte, können folgende Operationen durchgeführt werden
    a. Das Erhöhen oder Verringern einer Zahl um einen bestimmten Wert

Lists

- **Schüssel**: todos
- **Wert**: [“Einkaufen gehen“, “Staubsaugen“, “ Rechnungen zahlen“]

Bei einer Liste können beliebig viele Zeichenketten gespeichert werden. Jeder Speicherkette wird ein bestimmter Index zugewiesen, unter welchem der Wert abrufbar ist.
Eine Liste stellt ebenso einige nützliche Operationen zur Verfügung. Die wichtigsten sind:

1. Elemente an den Anfang oder das Ende einer Liste hinzufügen
2. Elemente vom Anfang oder vom Ende zu entfernen
3. Sich einen bestimmten Bereich der Liste ausgeben zu lassen

Sets

- **Schlüssel**: kategorien
- **Wert**: [“Fiktion“, “ Superhelden“, “Horror“]

Sets sind in ihrer Funktionsweise stark mit der Liste verbunden. Der Hauptunterschied liegt darin, dass in einem Set keine Duplikate erlaubt sind.
Weitere Unterschiede liegen in der Art und Weise wie auf Werte innerhalb des Sets zugegriffen wird. Diese werden durch die Beispiele im Abschnitt *Praxis -> Grundlegende Redis Befehle* deutlich.

Hashes

- **Schlüssel**: user
- **Wert**: [“name“: “Max Mustermann“, “alter“: “42“, “email“: “max@mustermann.de“]

Hashes sind in Redis wie JSON-Objekte aufgebaut, bestehend aus einem Schlüssel und einem oder mehreren Feldern und Werten. Im Gegensatz zu JSON-Objekten können Hashes jedoch nicht verschachtelt werden.

Schlüssel mit Lebensdauer

Da Redis grundsätzlich für die Funktion als Cache und Zwischenspeicher konzipiert ist, können Schlüssel mit einer TTL (Time-to-Live) versehen werden. Somit können Daten für

einen bestimmten Zeitraum zwischengespeichert oder gecached werden. Auch hier wird in dem Kapitel *Praxis -> Grundlegende Redis Befehle* weiter drauf eingegangen.

Praxis

Installation

Für die Durchführung der Übungsaufgaben werden ein paar Programme benötigt. Im Folgenden erfahren Sie, wie sie diese für die gängigen Systeme installieren.

Node.js

Die Installation von Node.js ist für alle gängigen Systeme sehr einfach. Sie müssen lediglich auf die offizielle Website gehen und die Installationsdatei herunterladen. Empfohlen wird die Installation der LTS. Laden Sie diese herunter und führen Sie die Installationsdatei aus.
Die Installationsdatei finden Sie unter: https://nodejs.org/en/download

Um zu überprüfen, ob die Installation erfolgreich war, können sie den Befehl *node -v* in einer Komandozeile ausführen. Wenn die Installation erfolgreich war, bekommen sie die Versionsnummer ihrer Installation ausgegeben.

Docker

Bei Docker hingegen ist die Installation je nach Betriebssystem etwas komplizierter. Unter Linux und Mac ist die Installation sehr leicht. Hier lässt sich die Installation über die Komandozeile durchführen oder über eine Installationsdatei. Dabei muss vorher nichts weiter eingestellt werden.

Unter Windows muss hier im Vorfeld betrachtet werden, welche Version von Windows installiert ist. Docker ist ursprünglich für den Linuxkernel entwickelt worden und benötigt diesen daher. Unter Linux ist dieser logischerweise vorhanden. Unter MacOS ist auch bereits ein Linuxkernel vorhanden. Windows besitzt von Haus aus allerdings keinen Linuxkernel. Allerdings kann Docker auch auf dem des Hyper-V backend laufen. Hier gibt es allerdings das Problem, dass das Hyper-V Feature der Windows Pro Version vorbehalten ist. Trotz dessen lässt sich Docker mittlerweile auch auf einem Windows Home System installieren. Hier muss dafür zunächst das Linux Subsystem installiert werden. Zusammengefasst muss daher folgendes vorab installiert/eingestellt werden.

- Home
  - Hier Home muss im Vorhinein das Linux Subsystem installiert werden
- Pro
  - Hier muss lediglich die Unterstützung von Hyper-V aktiviert werden

Nach der Vorbereitung kann auch hier die Installationsdatei heruntergeladen und ausgeführt werden.
Eine explizierte Anleitung für ihr Betriebssystem finden sie in der Dokumentation von Docker. Auch speziell für verschiedene Linux Systeme.

https://docs.docker.com/engine/install/

Einführung in Docker und Erstellung einer Redis-Datenbank

Docker ist eine Plattform, die es ermöglicht Anwendungen in Containern zu entwickeln, zu verteilen und auszuführen. Ein Container ist eine isolierte Umgebung, die die Anwendung und ihre Abhängigkeiten enthält. Der Container läuft auf einem eigenen Betriebssystem. Somit kann man sich diesen als virtuellen Computer vorstellen. Dadurch muss das eigene System nicht angefasst werden. Das bietet den Vorteil, dass man viele verschiedene Programme und Services testen kann, ohne sein eigenes System anzufassen. Darüber hinaus bietet Docker folgende weitere Vorteile:

- **Vereinfachte Bereitstellung von Anwendungen:** Mit Docker können Entwickler Anwendungen in einer konsistenten Umgebung entwickeln und diese dann nahtlos auf andere Umgebungen übertragen. Dies bedeutet, dass Entwickler sich nicht mit den unterschiedlichen Konfigurationen von Produktions- und Entwicklungsservern auseinandersetzen müssen.
- **Isolation von Anwendungen durch Container:** Container bieten eine isolierte Umgebung für Anwendungen, die die Abhängigkeiten und Konfigurationen der Anwendung enthält. Dadurch können mehrere Anwendungen auf einem Host-Betriebssystem ausgeführt werden, ohne dass sie sich gegenseitig beeinflussen oder stören.
- **Bessere Auslastung von Ressourcen:** Container können auf einem Host-Betriebssystem ausgeführt werden, ohne dass jeder Container ein eigenes Betriebssystem benötigt. Dadurch wird die Auslastung von Ressourcen wie Speicherplatz und CPU verbessert.
- **Schnellere Bereitstellung von Anwendungen:** Docker ermöglicht es Entwicklern Anwendungen schnell bereitzustellen, da sie in Containern verpackt sind, die leicht zu verteilen und zu skalieren sind.
- **Portabilität von Anwendungen:** Container sind portabel, was bedeutet, dass sie auf jedem Host-Betriebssystem ausgeführt werden können, das Docker unterstützt. Dadurch wird die Entwicklung und Bereitstellung von Anwendungen einfacher und schneller.

Für die Übungsaufgaben werden nur die grundlegenden Funktionalitäten von Docker benötigt. Daher werden wir uns im Folgenden auf dieses beschränken. Genauer gesagt beschränken wir uns auf die Erstellung eines Containers, welcher einen Redis-Server zur Verfügung stellen soll.

Jeder Container wird auf der Grundlage eines Images erstellt. Ein Image ist eine ausführbare Datei mit allen Anwendungsabhängigkeiten und -konfigurationen eines virtuellen Computers. Im Falle eines Redis-Servers, sind in dem Redis-Image alle nötigen Konfigurationen und Abhängigkeiten vordefiniert, die unser virtueller Computer oder auch Container braucht.

Wie bereits erwähnt funktioniert in Docker alles mit Containern. Diese sind vorstellbar, wie ein eigener virtueller Computer, mit Schnittstellen nach außen.
Die wichtigsten Befehle in der Komandozeile sind:

- **docker run -d --name redisDatenbank redis**
    - Das Image, welches als Container installiert werden soll, wird als letztes angegeben.
    - Bei diesem Befehl sucht Docker lokal nach dem Image Redis. Findet er dieses nicht, lädt er es herunter

    - Anschließend erstellt er auf Grundlage dieses Images einen Container und startet ihn
    - Mit --name können wir dem Container einen Namen geben
    - Der Befehl -d sorgt dafür das der Container im Hintergrund ausgeführt wird
- **docker ps**
    - Gibt eine Liste aller laufenden Container aus
- **docker stop <name>**
    - Stoppt einen laufenden Container
- **docker rm <name>**
    - Löscht einen Container

Für den Befehl docker run gibt es noch weitere Befehle die angehängt werden können. Um auf den Container von außen zugreifen zu können, muss ein Port definiert werden. Dies geschieht mit dem Befehl -p <Port Außen>:<Port Innen>.
Der Standartport für redis ist 6379. Daher verwenden wir diesen Port sowohl für nach innen, als auch nach außen.

docker run -d --name redisDatenbank -p 6379:6379 redis

An dieser Stelle müssen wir nun lediglich noch eine Konfiguration vornehmen. Denn jeder Container ist nicht persistent. Bedeutet nach einem Neustart des Containers sind alle Daten weg. Dies können wir verhindern, indem wir ein Verzeichnis in den Container „Mounten“. Das bedeutet, dass wir die Daten in der Datenbank lokal auf unserem System in einem Verzeichnis speichern und dieses dann beim Start des Containers übergeben. Das erfolgt mit dem Befehl: -v <Pfad zum lokalen Verzeichnis>:<Pfad zum Verzeichnis des Containers>. Die Pfade für die Container sind meist vorgeschrieben und können in der Dokumentation des Images nachgelesen werden. In unserem Fall sähe das exemplarisch wie folgt aus:

docker run -d --name redisDatabase -v /opt/docker/data/redis:/data -p 6379:6379 redis

Somit haben wir einen Container erstellt dessen Daten persistent sind und auf den wir von außen zugreifen können.

Grundlegende Redis Befehle

Um die Folgenden Befehle ohne spezifische Anwenung auszuführen, ist es möglich auf die Redis-CLI mittels Komandozeile zuzugreifen. Hierfür muss der entsprechende Docker Container laufen. Anschließend kann man mit dem Befehl: **docker exec it- <Datenbank name> <Pfad zur Bash>** auf den Container zugreifen.

In unserem Fall würde der Befehl wie folgt aussehen:
docker exec -it redisDatabase /bin/bash

Anschließend kann mit dem Befehl die CLI von Redis geöffnet werden:
redis-cli

Um die cli zu verlassen kann der Befehl **quit** genutzt werden, und um die Bash vom Container zu verlassen **quit**.

In dieser CLI können nun alle Befehle für Redis ausgeführt werden. Die wichtigsten Befehle werden im nächsten Kapitel dargelegt und mit Beispielen erläutert.

Strings + Allgemeine Befehle

DEL

Der Befehl `DEL` löscht einen Schlüssel und seinen Wert aus Redis.

Syntax: `DEL key`

Beispiel:

```
SET mykey "Hello Redis"
DEL mykey
```

Das obige Beispiel löscht den Schlüssel `mykey` und seinen Wert aus Redis. Wenn der Schlüssel nicht vorhanden ist, hat der Befehl keine Auswirkungen.

MGET

Der Befehl `MGET` gibt die Werte mehrerer Schlüssel in Redis zurück.

Syntax: `MGET key [key ...]`

Beispiel:

```
MSET name "Max" age 25 email "max@example.com"
MGET name age email
```

Das obige Beispiel gibt die Werte der Schlüssel `name`, `age` und `email` zurück, also `"Max"`, `25` und `"max@example.com"`.

SETNX

Der Befehl `SETNX` speichert einen Wert unter einem Schlüssel in Redis, wenn dieser Schlüssel noch nicht existiert.

Syntax: `SETNX key value`

Beispiel:

```
SETNX mykey "Hello Redis"
```

Das obige Beispiel speichert den Wert `"Hello Redis"` unter dem Schlüssel `mykey`, wenn dieser Schlüssel noch nicht existiert. Wenn der Schlüssel bereits existiert, hat der Befehl keine Auswirkungen. Der Rückgabewert des Befehls gibt an, ob der Schlüssel erfolgreich gesetzt wurde. Wenn der Schlüssel erfolgreich gesetzt wurde, gibt Redis `1` zurück. Wenn der Schlüssel bereits existierte, gibt Redis `0` zurück.

Dieser Befehl kann nützlich sein, wenn Sie sicherstellen möchten, dass ein Schlüssel nur einmal in Redis gesetzt wird.

DECRBY

Der Befehl `DECRBY` dekrementiert den Wert eines Schlüssels in Redis um einen bestimmten Wert.

Syntax: `DECRBY key decrement`

Beispiel:

```
SET counter 10
DECRBY counter 5
```

Das obige Beispiel dekrementiert den Wert des Schlüssels `counter` in Redis um `5`. Wenn der Schlüssel `counter` noch nicht existiert, wird er mit dem Wert `0` erstellt und anschließend dekrementiert.

DECR

Der Befehl `DECR` dekrementiert den Wert eines Schlüssels in Redis um 1.

Syntax: `DECR key`

Beispiel:

```
SET counter 10
DECR counter
```

INCRBY

Der Befehl `INCRBY` inkrementiert den Wert eines Schlüssels in Redis um einen bestimmten Wert.

Syntax: `INCRBY key increment`

Beispiel:

```
SET counter 0
INCRBY counter 5
```

Das obige Beispiel inkrementiert den Wert des Schlüssels `counter` in Redis um `5`. Wenn der Schlüssel `counter` noch nicht existiert, wird er mit dem Wert `0` erstellt und anschließend inkrementiert.

INCR

Der Befehl `INCR` inkrementiert den Wert eines Schlüssels in Redis um 1.

Syntax: `INCR key`

Beispiel:

```
SET counter 0
INCR counter
```

KEYS

Der Befehl `KEYS` gibt eine Liste aller Schlüssel zurück, die einem bestimmten Muster entsprechen.

Syntax: `KEYS pattern`

Beispiel:

```
SET name "Max"
SET age 25
SET location "Berlin"
KEYS *name*
```

Das obige Beispiel gibt alle Schlüssel zurück, die das Wort "name" im Schlüssel enthalten, also `"name"`.

Wenn Sie jedoch den Befehl `KEYS *` ausführen, werden alle Schlüssel im Redis-Speicher zurückgegeben, was bei großen Datenmengen sehr zeitaufwändig sein kann und die Leistung beeinträchtigen kann. Daher wird empfohlen die Muster als Argument möglichst spezifisch zu halten.

EXISTS

Der Befehl `EXISTS` prüft, ob ein Schlüssel in Redis existiert.

Syntax: `EXISTS key`

Beispiel:

```
SET mykey "Hello Redis"
EXISTS mykey
```

Das obige Beispiel gibt `1` zurück, wenn der Schlüssel `mykey` in Redis vorhanden ist, andernfalls gibt es `0` zurück.

GET

Der Befehl `GET` gibt den Wert eines Schlüssels in Redis zurück. Dieser Befehl funktioniert nur für Strings

Syntax: `GET key`

Beispiel:

```
GET mykey
```

SET

Der Befehl `SET` speichert einen Wert unter einem Schlüssel in Redis.

Syntax: `SET key value`

Beispiel:

```
SET mykey "Hello Redis"
```

GET

Der Befehl `GET` gibt den Wert eines Schlüssels in Redis zurück. Dieser Befehl funktioniert nur für Strings

Syntax: `GET key`

Beispiel:

```
GET mykey
```

Ablaufende Schlüssel

TTL

Der Befehl `TTL` gibt die verbleibende Lebensdauer eines Schlüssels in Sekunden zurück. Wenn der Schlüssel nicht abläuft, gibt Redis `-1` zurück. Wenn der Schlüssel nicht vorhanden ist, gibt Redis `-2` zurück.

Syntax: `TTL key`

Beispiel:

```
SET mykey "Hello Redis"
EXPIRE mykey 60
TTL mykey
```

Das obige Beispiel speichert den Schlüssel `mykey` mit dem Wert `"Hello Redis"` und einer Lebensdauer von 60 Sekunden. Der Befehl `TTL mykey` gibt die verbleibende Lebensdauer des Schlüssels `mykey` in Sekunden zurück. Wenn der Schlüssel noch nicht abgelaufen ist, gibt Redis die verbleibende Zeit in Sekunden zurück. Wenn der Schlüssel nicht vorhanden ist, gibt Redis `-2` zurück.

SETEX

Der Befehl `SETEX` speichert einen Schlüssel mit einem bestimmten Wert und einer ablaufenden Lebensdauer in Redis.

Syntax: `SETEX key seconds value`

Beispiel:

```
SETEX mykey 60 "Hallo Redis"
```

Das obige Beispiel speichert den Schlüssel `mykey` mit dem Wert `"Hallo Redis"` und einer Lebensdauer von 60 Sekunden. Das bedeutet, dass der Schlüssel und sein Wert nach 60 Sekunden automatisch aus Redis gelöscht werden.

EXPIRE

Der Befehl `EXPIRE` legt die Lebensdauer eines Schlüssels in Redis fest und gibt an, wie lange der Schlüssel im Speicher bleiben soll, bevor er automatisch gelöscht wird.

Syntax: `EXPIRE key seconds`

Beispiel:

```
SET mykey "Hello Redis"
EXPIRE mykey 60
```

Das obige Beispiel legt die Lebensdauer des Schlüssels `mykey` auf 60 Sekunden fest. Nach Ablauf dieser Zeit wird der Schlüssel und sein Wert automatisch aus Redis gelöscht. Wenn Sie die Lebensdauer eines Schlüssels verlängern möchten, können Sie den Befehl `EXPIREAT` verwenden, der eine UNIX-Zeitstempel anstelle von Sekunden akzeptiert.

```
EXPIREAT mykey 1627992000
```

Dieses Beispiel legt die Lebensdauer des Schlüssels `mykey` auf den 3. August 2021 um 12:00 Uhr UTC fest. Nach Ablauf dieser Zeit wird der Schlüssel und sein Wert automatisch aus Redis gelöscht.

Es ist wichtig zu beachten, dass der Befehl `EXPIRE` nur auf Schlüssel angewendet werden kann, die bereits existieren. Wenn der Schlüssel nicht vorhanden ist, hat der Befehl keine Auswirkungen.

Lists

RPOP

Der Befehl `RPOP` entfernt und gibt das letzte Element aus einer Liste in Redis zurück.

Syntax: `RPOP key`

Beispiel:

```
RPUSH mylist "hello"
RPUSH mylist "world"
RPOP mylist
```

LRANGE

Der Befehl `LRANGE` gibt eine Teilmenge einer Liste in Redis zurück. Sie müssen einen Start- und einen Endindex angeben, um die Elemente auszuwählen, die Sie in der Liste anzeigen möchten.

Syntax: `LRANGE key start end`

Beispiel:

```
LPUSH mylist "world"
LPUSH mylist "hello"
LRANGE mylist 0 1
```

Das obige Beispiel gibt die ersten beiden Elemente in der Liste `mylist` zurück, also `"hello"` und `"world"`.

Um sich die gesamte Liste ausgeben zu lassen, geben Sie als Startindex 0 und als Endindex -1 an.

LPUSH

Der Befehl `LPUSH` fügt ein Element am Anfang einer Liste in Redis hinzu.

Syntax: `LPUSH key value`

Beispiel:

```
LPUSH mylist "world"
LPUSH mylist "hello"
```

RPUSH

Der Befehl `RPUSH` fügt ein Element am Ende einer Liste in Redis hinzu.

Syntax: `RPUSH key value`

Beispiel:

```
RPUSH mylist "hello"
RPUSH mylist "world"
```

LPOP

Der Befehl `LPOP` entfernt und gibt das erste Element aus einer Liste in Redis zurück.

Syntax: `LPOP key`

Beispiel:

```
LPUSH mylist "world"
LPUSH mylist "hello"
LPOP mylist
```

Sets

SREM

Der Befehl `SREM` entfernt Mitglieder aus einem Set in Redis.

Syntax: `SREM key member [member ...]`

Beispiel:

```
SADD myset "Hello"
SADD myset "World"
SADD myset "Redis"
SREM myset "World"
```

Das obige Beispiel entfernt das Mitglied `"World"` aus dem Set `myset`. Das Set enthält nun nur noch die Mitglieder `"Hello"` und `"Redis"`.

SADD

Der Befehl `SET` speichert einen Wert unter einem Schlüssel in Redis. Im Unterschied zu einer Liste in Redis, können in einem Set keine Duplikate von Werten gespeichert werden.

Syntax: `SADD key member [member ...]`

Beispiel:

```
SADD myset "Hello"
SADD myset "World"
```

SMEMBERS

Der Befehl `SMEMBERS` gibt alle Mitglieder eines Sets in Redis zurück.

Syntax: `SMEMBERS key`

Beispiel:

```
SADD myset "Hello"
SADD myset "World"
SMEMBERS myset
```

Das obige Beispiel gibt alle Mitglieder des Sets `myset` zurück, also `"Hello"` und `"World"`.

Hashes

HEXISTS

Der Befehl `HEXISTS` gibt zurück, ob ein Feld in einem Hash-Objekt existiert.

Syntax: `HEXISTS key field`

Beispiel:

```
HEXISTS user name
```

Das obige Beispiel gibt `1` zurück, wenn das Feld `name` im Hash-Objekt `user` vorhanden ist, andernfalls gibt es `0` zurück.

HDEL

Der Befehl `HDEL` löscht ein Feld in einem Hash-Objekt.

Syntax: `HDEL key field`

Beispiel:

```
HDEL user age
```

Das obige Beispiel löscht das Feld `age` aus dem Hash-Objekt `user`.

HSET

Der Befehl `HSET` fügt einem Hash-Objekt ein Feld mit einem Wert hinzu.

Syntax: `HSET key field value`

Beispiel:

```
HSET user name "Max"
HSET user age 25
```

HGET

Der Befehl `HGET` gibt den Wert eines Feldes in einem Hash-Objekt zurück.

Syntax: `HGET key field`

Beispiel:

Plain Text Kopieren Beschriftung

```
HGET user name
```

HGETALL

Der Befehl `HGETALL` gibt alle Felder und Werte eines Hash-Objekts zurück.

Syntax: `HGETALL key`

Beispiel:

```
HSET user name "Max"
HSET user age 25
HGETALL user
```

Das obige Beispiel gibt alle Felder und Werte des Hash-Objekts `user` zurück, also `"name"` und `"Max"` sowie `"age"` und `25`.

Weitere Befehle

FLUSHALL

Der Befehl `FLUSHALL` löscht alle Schlüssel und ihre Werte aus allen Datenbanken in Redis. Achtung: Dieser Befehl kann nicht rückgängig gemacht werden.

Syntax: `FLUSHALL`

Beispiel:

Plain Text Kopieren Beschriftung

```
SET mykey "Hello Redis"
FLUSHALL
```

Das obige Beispiel löscht den Schlüssel `mykey` und alle anderen Schlüssel und Werte in Redis.

Literatur

- https://www.mysql.com/customers/
- https://www.oracle.com/industries/financial-services/
- https://www.mongodb.com/industries/financial-services
- https://docs.docker.com/engine/install/
- https://www.datenbanken-verstehen.de/lexikon/key-value-datenbanksysteme/
- https://www.youtube.com/watch?v=WhCyMGpR8mM
- https://insightsoftware.com/blog/whats-the-difference-relational-vs-non-relational-databases/#:~:text=To
- https://www.tutorialspoint.com/redis/redis_commands.htm